

Minimización de Funciones Booleanas

La simplificación de funciones mediante el álgebra booleana puede ser una tarea difícil, ya que no existe un procedimiento claro a seguir: uno debe buscar el camino para la simplificación en base a la intuición y la experiencia. Para realizar la simplificación de funciones eficientemente, presentaremos un método llamado **Mapas de Karnaugh**, el cual es un método gráfico que proporciona una técnica estándar y sistemática en la reducción manual de funciones booleanas.

Hasta 4 2^n

Rango óptimo del Mapa de Karnaugh

Tamaño del mapa para n variables

100%

Método gráfico sin ambigüedad

$$F = \sum m(m_0, m_1, \dots, m_{2^n-1})$$

LÓGICA DIGITAL

ÁLGEBRA BOOLEANA

DISEÑO DE CIRCUITOS



¿Por qué necesitamos minimizar?

La optimización es crítica en el diseño de software y hardware. El objetivo es **reducir el costo computacional y la complejidad de las expresiones lógicas**. Una vez que obtenemos la forma canónica (Suma de Productos), a menudo nos queda una expresión muy larga y costosa de procesar. Necesitamos simplificarla.

Álgebra Booleana

Podemos usar álgebra aplicando las leyes del álgebra booleana, pero es propenso a errores humanos.

Mapas de Karnaugh

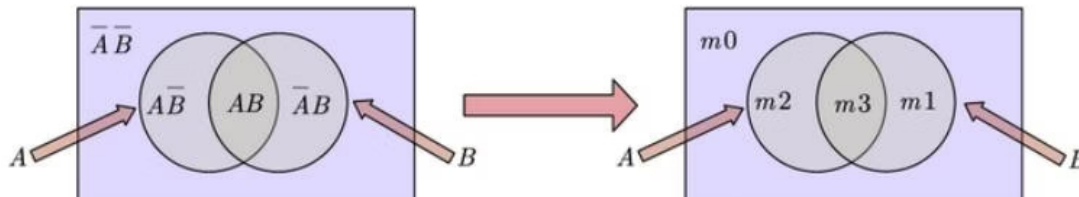
Para funciones de hasta 4 variables, existe una herramienta gráfica visualmente infalible: el **Mapa de Karnaugh (K-Map)**. Es la solución estándar y sistemática.

- ❏ La optimización impacta directamente en el número de compuertas lógicas necesarias, reduciendo el costo de fabricación y el consumo energético de los circuitos integrados.

¿Cómo funciona el Mapa de Karnaugh?

Los Mapas de Karnaugh (MK) no son más que una extensión de los conceptos de tablas de verdad, diagramas de Venn y minterminos. A continuación se ilustrará la manera en que estos tres conceptos se entrelazan para obtener lo que llamaremos un Mapa de Karnaugh.

Para ello, consideremos el Diagrama de Venn de dos conjuntos A y B y localicemos en él los subconjuntos o regiones correspondientes a los cuatro minterminos $\bar{A}\bar{B}$, $A\bar{B}$, AB y $\bar{A}B$, es decir, los minterminos m_0 , m_1 , m_2 y m_3 .



Como puede verse, todo el diagrama de Venn se puede particionar en estas cuatro regiones independientes (no tienen puntos en común) y cada región está identificada por un mintermino. El diagrama muestra la transformación de un Diagrama de Venn a un Mapa de Karnaugh: a la izquierda, un diagrama de Venn con dos círculos A y B; a la derecha, un Mapa de Karnaugh con las mismas cuatro regiones, pero ahora son celdas adyacentes en un cuadrado. Por otro lado, nada nos obliga a dibujar los conjuntos A y B redondos y el conjunto universo cuadrado.

Organización del Mapa: El Código Gray

El Mapa de Karnaugh es una grilla donde las celdas se ordenan de tal manera que las celdas adyacentes solo difieren en el valor de una sola variable. Esto se llama **Código Gray** y es el fundamento del método.

Mapa de 2 variables
 $2^2 = 4$ celdas. Variables A (filas) y B (columnas). Orden: 0, 1.

Mapa de 3 variables
 $2^3 = 8$ celdas. Variable A (filas), BC (columnas). Orden Gray: 00, 01, 11, 10.

Mapa de 4 variables
 $2^4 = 16$ celdas. Variables AB (filas), CD (columnas). Orden Gray en ambos ejes.

Mapa 2 variables

A \ B	0	1
0	m0	m1
1	m2	m3

Mapa 3 variables

X \ YZ	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

Mapa 4 variables

AB \ CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

El número de cuadrados en un mapa de n variables es 2^n minterminos, listados por un número decimal equivalente para fácil referencia. Los números de minterminos son asignados en un arreglo ordenado de tal manera que los cuadrados adyacentes representen los minterminos que difieren solamente en una variable. El orden de las combinaciones no es binario natural, sino que es **código Gray (00, 01, 11, 10)**, esto debido a que el funcionamiento del método se basa en combinaciones adyacentes.

 **Propiedad clave:** Dos celdas son adyacentes si y solo si difieren en exactamente una variable. Esta propiedad de distancia unitaria es lo que permite eliminar variables redundantes al agrupar celdas.

Representación en Mapas de Karnaugh: Ejemplos

A continuación se muestran algunos ejemplos de representación en mapa de Karnaugh de distintas funciones a partir de su tabla de verdad. Este mapa representará los valores a minimizar cuando la función arroje un valor igual a 1.

a) Mapa de Karnaugh para dos variables

x	y	F(x, y)
0	0	0
0	1	1
1	0	0
1	1	1

	y=0	y=1
x=0	0	1
x=1	2	3

b) Mapa de Karnaugh para tres variables

x	y	z	F(x,y,z)
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

YZ\X	00	01	11	10
0	0 (m0)	1 (m1)	0 (m3)	1 (m2)
1	1 (m4)	1 (m5)	1 (m7)	0 (m6)

Identificamos los grupos de unos adyacentes en el mapa:

Grupo A: {m4, m5}

X=1, Z varía, Y=0 → término XY'

Grupo B: {m1, m5}

Z=1, Y=0, X varía → término Y'Z

Grupo C: {m2}

X=0, Y=1, Z=0 → término X'YZ'

$$F(X,Y,Z) = XY' + Y'Z + X'YZ'$$

Mapa de Karnaugh para Cuatro Variables

c) Ejemplo completo con tabla de verdad y mapa

A	B	C	D	F(A,B,C,D)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1

AB\CD	00	01	11	10
00	0 (m0)	1 (m1)	1 (m3)	0 (m2)
01	0 (m4)	1 (m5)	1 (m7)	0 (m6)
11	0 (m12)	1 (m13)	1 (m15)	0 (m14)
10	0 (m8)	1 (m9)	1 (m11)	0 (m10)

Los nombres de las variables son listados a través de ambos lados de la línea diagonal en la esquina del mapa. Los 1 y 0 marcados a lo largo de cada fila y de cada columna designan el valor de las variables.

En este mapa, los unos se encuentran en todas las posiciones donde D=1 (columnas 01 y 11). Esto significa que la variable D es la única que determina la salida:

$$F(A,B,C,D) = D$$

- ☐
- Este es un caso especial donde la función depende de una sola variable. El mapa muestra una columna completa de unos, lo que indica que sin importar los valores de A, B y C, la salida siempre es igual a D. Esto demuestra el poder del Mapa de Karnaugh: de 16 minterminos a una sola variable.

Forma canónica (SOP)

F = A'B'C'D + A'B'CD + A'BC'D + A'BCD + AB'C'D + AB'CD + ABC'D + ABCD (8 términos, 32 literales)

Forma mínima

F = D (1 término, 1 literal). ¡Reducción del 97%!

Metodología de Reducción: Proceso de Minimización

Vamos a indicar cada uno de los pasos para obtener la expresión SOP (mínima suma de productos). Para ello vamos a ilustrarlo con el ejemplo:

$$F(x,y,z) = x'y'z' + x'y'z + x'yz' + xyz' + xyz$$

01

Convertir a Suma de Productos

Esto se puede realizar algebraicamente, o construyendo una tabla de verdad y trasladando los valores al mapa de Karnaugh. Esta es la forma que vamos a utilizar.

02

Identificar lazos de 2^N unos

Cubrir todos los unos del mapa mediante rectángulos de 2^N elementos, donde $N = 0 \dots$ número de variables. Ninguno de esos rectángulos debe contener ningún cero.

03

Maximizar el tamaño de cada rectángulo

Para minimizar el número de términos resultantes se hará el mínimo número posible de rectángulos que cubran todos los unos. Para minimizar el número de variables se hará cada rectángulo tan grande como sea posible.

04

Encontrar la MSP (suma de productos mínima)

Cada rectángulo pertenece a un término producto. Cada término se define encontrando las variables comunes en cada rectángulo. Atención: puede haber más de una MSP.

Ejemplo Paso a Paso de Minimización

Aplicamos la metodología a $F(x, y, z) = x'y'z' + x'y'z + x'yz' + xy'z' + xyz'$. Primero construimos la tabla de verdad:

x	y	z	Resultado
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

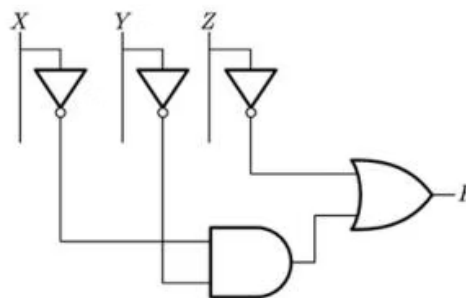
Mapa de Karnaugh resultante

$\begin{matrix} x & y \\ z \end{matrix}$	00	01	11	10
0	1	1	1	1
1	1	0	0	0

Se unen las celdas con valor 1 para formar rectángulos, en cada caso **el más grande posible**. En el rectángulo azul el elemento común es Z' , y en el rectángulo rojo los elementos comunes son $X'Y'$.

Por lo tanto la función resultante es:

$$F(X, Y, Z) = X'Y' + Z'$$



Rectángulos, Productos e Implicantes Primos

Cada rectángulo representa un término. El tamaño del rectángulo y el del término resultante son **inversamente proporcionales**: cuanto más largo sea el rectángulo, menor será el tamaño del término final.

En general, si tenemos una función con n variables:

1 celda

Un rectángulo que ocupa **una celda** equivale a un término con **n variables**.

2 celdas

Un rectángulo que ocupa **dos celdas** equivale a un término con **$n-1$ variables**.

2^n celdas

Un rectángulo que ocupa 2^n **celdas** equivale al término de valor **1**.

Agrupación de rectángulos: Implicantes Primos

Cuando tenemos distintas posibilidades de agrupar rectángulos hay que seguir ciertos criterios:

- Localiza todos los rectángulos más grandes posibles, agrupando todos los unos. Estos se llamarán **implicantes primos**.
- Si alguno de los rectángulos anteriores contiene algún uno que no aparece en ningún otro rectángulo, entonces es un **implicante primo esencial**. Estos han de aparecer en el resultado final de manera obligatoria.
- El resto de implicantes primos se podrán combinar para obtener distintas soluciones.

 **Regla de eliminación de variables redundantes:** Cuando dos celdas adyacentes se agrupan, la variable que cambia entre ellas se elimina del término resultante. Solo permanecen las variables que son comunes (constantes) en todo el grupo.

Ejemplo: Tamaño de rectángulo vs. variables eliminadas

Tamaño del grupo	Celdas	Variables eliminadas	Término resultante ($n=3$)
1 celda	$2^0 = 1$	0 variables	3 literales (ej: X'Y'Z')
2 celdas	$2^1 = 2$	1 variable	2 literales (ej: X'Y')
4 celdas	$2^2 = 4$	2 variables	1 literal (ej: X')
8 celdas	$2^3 = 8$	3 variables	Constante 1

$$\text{Variables en término} = n - \log_2(\text{tamaño del grupo})$$

Ejemplo de Implicantes Primos Esenciales

Véase este ejemplo que ilustra los implicantes primos. Aquí los implicantes primos son cada uno de los diferentes rectángulos obtenidos.

AB\CD	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	0	1	1	0
10	0	0	1	1

Los **primos implicantes esenciales** son el rectángulo azul y el verde, por contener unos que no son cubiertos por otros rectángulos. Así, todas las posibles soluciones han de contener estos dos implicantes. Luego descartamos los rectángulos amarillo y morado para obtener como tercer componente de la solución el rectángulo rojo.

Solución: $F(A, B, C, D) = ABD + AB'C + A'C'$

Para encontrar el MSP se debe:

- **Minimizar** el número de rectángulos que se hacen en el mapa de Karnaugh, para minimizar el número de términos resultantes.
- **Maximizar** el tamaño de cada rectángulo, para minimizar el número de variables de cada término resultante.

Condiciones de Indiferencia (Don't Care)

Además de los conceptos de implicantes primos y esenciales, existe un recurso adicional muy poderoso para lograr una simplificación aún mayor: las **Condiciones de Indiferencia** (en inglés, *Don't Care*).

En muchos sistemas reales, ciertas combinaciones de entrada **nunca ocurren** o simplemente **no importa** cuál sea la salida para esas combinaciones. Estas condiciones se representan con el símbolo X en la tabla de verdad y en el mapa de Karnaugh.



¿Qué son?

Estados no definidos de la función, marcados con X en el mapa. Pueden tratarse como 0 o como 1, según convenga para la minimización.



¿Cómo se usan?

Al formar los lazos (rectángulos), se puede incluir una celda X si eso permite agrandar el grupo y así eliminar más variables del término resultante.



Regla clave

No es obligatorio cubrir las celdas X . Solo se incluyen cuando ayudan a formar un rectángulo más grande. Nunca se deben cubrir solo para completar grupos.

- Las condiciones de indiferencia son especialmente frecuentes en sistemas con codificación BCD (Binary Coded Decimal), donde las combinaciones del 10 al 15 nunca se utilizan y pueden marcarse como X .

Ejemplo: Función BCD con condiciones de indiferencia

Consideremos una función $F(A,B,C,D)$ definida sobre dígitos BCD (0–9). Las combinaciones del 10 al 15 nunca ocurren en BCD, por lo que se marcan como X (don't care).

AB\CD	00	01	11	10
00	1	0	X	1
01	0	1	X	X
11	X	X	X	X
10	1	1	X	0

Sin Don't Care

$$F = A'B'C'D' + A'B'CD + \dots$$

Sin aprovechar las X , obtenemos 4 términos.

Con Don't Care

$$F = B'D' + A'CD$$

Aprovechando las X como 1s, reducimos a solo 2 términos.

- Las celdas X se pueden incluir en un grupo si agrandan el rectángulo, pero nunca son obligatorias de cubrir.

El Procedimiento de Diseño de Circuitos Lógicos

Aunque en principio, al abordar el problema de diseño de un circuito podemos usar nuestra experiencia, intuición, inventiva y sentido común, todos estos elementos no nos garantizan una solución óptima. Sin pretender coartar la creatividad e imaginación del estudiante, se propone el siguiente **Procedimiento de Diseño de Circuitos Lógicos**:

- 1

Identificar variables y Diagrama de Bloques
Identificar en el enunciado del problema las variables involucradas en el diseño, asignarles un símbolo y representarlas en un Diagrama de Bloques, el cual es un bloque desconocido interiormente aún, pero su exterior debe estar bien especificado: un conjunto de entradas (variables independientes) y salidas (variables dependientes o funciones a diseñar).
- 2

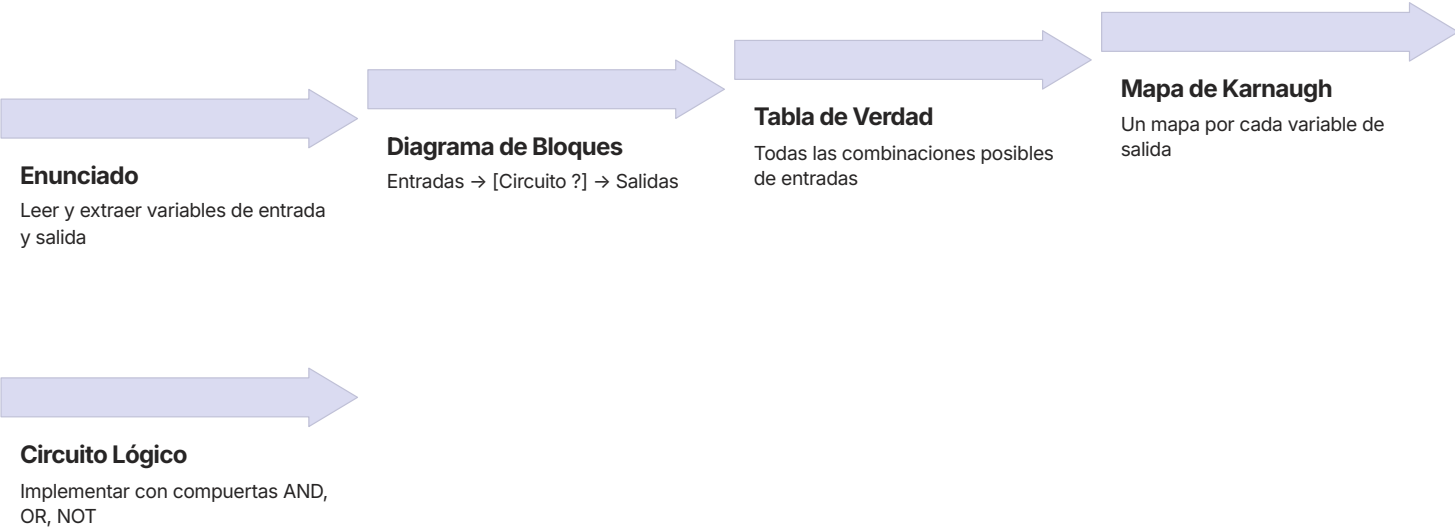
Tabla de Verdad
Interpretar en el enunciado las relaciones de dependencia lógica entre entradas y salidas y representarlas en una Tabla de Verdad. Este paso debe realizarse cuidadosamente, dado la ambigüedad propia del lenguaje que puede llevar a relaciones lógicas equivocadas. No debe olvidarse la relación directa de los conectivos gramaticales Y, O con las operaciones AND, OR respectivamente, así como la negación gramatical con el complemento lógico (NOT). Hay que ser cuidadoso de las interpretaciones gramaticales de O inclusivo (OR) y O exclusivo (EXOR).
- 3

Mapas de Karnaugh
Obtener de la tabla de verdad los Mapas de Karnaugh, **un mapa por cada variable de salida**.
- 4

Expresiones lógicas mínimas
Obtener las expresiones lógicas mínimas para cada variable de salida usando los mapas del paso anterior.
- 5

Implementación con puertas lógicas
Dibujar la implementación con puertas lógicas procurando usar el mínimo de circuitos integrados. Una expresión lógica mínima NO garantiza una implementación mínima en términos de circuitos integrados. Se puede recurrir al uso de puertas lógicas de un solo tipo (NAND o NOR): las NAND son adecuadas para formas SP y las NOR para formas PS.

Resumen Visual del Procedimiento



❏ Recuerda: una expresión lógica mínima NO garantiza una implementación mínima en circuitos integrados. Siempre evalúa si conviene usar compuertas NAND (para SOP) o NOR (para POS) para reducir el número de chips necesarios.

Ejemplo Completo: Control de Encendido de Lámpara

Diseñar un circuito para controlar el encendido y apagado de una lámpara, la cual deberá encender siempre que sea de noche, o la iluminación ambiental sea muy baja, a menos que se desee ahorrar energía.

Señal N

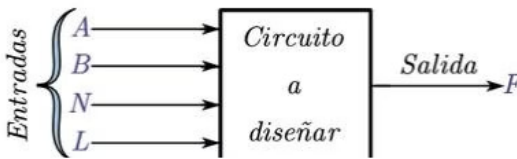
Indicador de horario nocturno, controlada por un reloj. **N = 1** indica que es de noche.

Señal L

Indicador de iluminación ambiental, controlada por una fotocelda. **L = 1** indica que aún hay luz ambiental.

Señales A, B

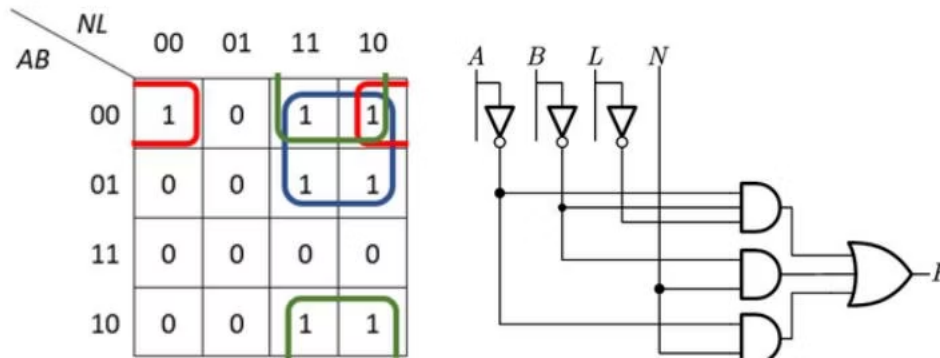
Par de conmutadores para indicar ahorro de energía: **AB = 00** no ahorra; un solo activado bloquea L; **AB = 11** la luz no se activa.



Mintérmino	A	B	N	L	F
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

Solución del Ejemplo: Mapa de Karnaugh y Función Simplificada

$$F(A, B, N, L) = \sum m(0, 2, 3, 6, 7, 10, 11)$$



De acuerdo con el M.K. anterior, obtenemos:

$$F = A'B'L' + A'N + B'N$$

- Los tres grupos identificados en el mapa corresponden a: el **grupo rojo** ($A'B'L'$): noche sin luz y sin ahorro; el **grupo azul** ($A'N$): de noche sin conmutador A; y el **grupo verde** ($B'N$): de noche sin conmutador B. Cada grupo es un implicante primo esencial.

Implementación con Compuertas Lógicas

La función $F = A'B'L' + A'N + B'N$ se implementa con compuertas AND, OR y NOT. Cada término producto requiere una compuerta AND, y los tres términos se combinan con una compuerta OR de 3 entradas.

Término $A'B'L'$

Compuerta AND de 3 entradas. Entradas: NOT(A), NOT(B), NOT(L). Activa cuando no hay ahorro y no hay luz ambiental.

Término $A'N$

Compuerta AND de 2 entradas. Entradas: NOT(A), N. Activa cuando es de noche y el conmutador A no está activado.

Término $B'N$

Compuerta AND de 2 entradas. Entradas: NOT(B), N. Activa cuando es de noche y el conmutador B no está activado.

$$F = (A' \cdot B' \cdot L') + (A' \cdot N) + (B' \cdot N)$$

- Total de compuertas necesarias: 3 inversores (NOT), 1 AND de 3 entradas, 2 AND de 2 entradas, 1 OR de 3 entradas = 7 compuertas. Sin minimización, la forma canónica requeriría más del doble.

Compuertas Lógicas, Circuitos Combinacionales y Reglas de Negocio

Hasta ahora hemos hablado de matemáticas abstractas. Pero la computadora es una máquina física hecha de silicio y electricidad. ¿Cómo "piensa" el silicio? A través de transistores organizados en **Compuertas Lógicas**. Una compuerta es un dispositivo físico que implementa una función booleana.

AND (Producto)
Forma de D mayúscula. La salida se enciende solo si *todas* las entradas tienen voltaje (1).

OR (Suma)
Forma de punta de flecha curva. La salida se enciende si *alguna* entrada tiene voltaje.

NOT (Inversor)
Triángulo con círculo. Invierte el voltaje: si entra corriente (1), sale tierra (0).

NAND y NOR
Compuertas universales. Más baratas de fabricar. ¡Tu procesador está hecho casi enteramente de millones de compuertas NAND! Cualquier función lógica se puede construir usando solo compuertas NAND.

XOR (O Exclusiva)
Fundamental para la criptografía y para sumar bits. Si tienes dos bits iguales da 0, si son distintos da 1.

Un conjunto de compuertas conectadas donde la salida depende únicamente de las entradas actuales (sin memoria del pasado) se llama **Circuito Combinacional**. Es la implementación física de nuestras funciones booleanas.

Caso de Estudio: Sistema Bancario

Como desarrollador, te enfrentarás a documentos de requisitos funcionales llamados "Reglas de Negocio". Requisito: *"Se aprobará el préstamo si el cliente tiene ingresos superiores a 1000 y antigüedad mayor a 2 años, o si es cliente VIP, pero en ningún caso se aprobará si figura en el veraz (lista de deudores)."*

Variables definidas

- I: Ingresos > 1000
- A: Antigüedad > 2
- V: Es VIP
- D: Está en lista de deudores

Función lógica original

Aprobado = ((I · A) + V) · ¬D

Fíjate en el paréntesis y el operador final. La condición ¬D (NO tener deuda) multiplica a todo lo anterior. Esto es un **"Kill Switch"** o condición de bloqueo. Si D = 1 (tiene deuda), ¬D = 0. Y cualquier cosa multiplicada por 0 es 0. El préstamo se deniega. Si la escribes mal, podrías dar préstamos a deudores VIP, causando pérdidas millonarias al banco. **La lógica booleana es tu seguro de calidad.**

Tablas de Verdad de las Compuertas Básicas

A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

AND: salida 1 solo si TODAS las entradas son 1

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

OR: salida 1 si AL MENOS UNA entrada es 1

A	NOT A
0	1
1	0

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

NOT invierte. XOR: 1 si las entradas son distintas

Las compuertas NAND y NOR son llamadas 'universales' porque cualquier función booleana puede implementarse usando únicamente uno de estos dos tipos de compuertas. Esto simplifica enormemente la fabricación de chips.